Step-by-Step Guide

Building a max-quality streaming Speech-to-Text platform — latency, managed scaling, diarization, enterprise reliability, accuracy tuning.

Source repo: true-streaming-stt-provider

Companion docs: README.md, docs/streaming-design.md, docs/maximize-each-category.md

Prepared: May 2026

> *This guide turns the strategy doc into an actionable sequence. Phase 0 decides which path you take; everything after Phase 1 forks into a cloud-first track (Phase 2A) and a self-hosted track (Phase 2B). Pick one. Pick both later if you need a hybrid.*

How to use this guide

Each phase below is a sequence of numbered steps. A step lists what to do, what to deliver, and a rough effort estimate for a 2–3 engineer team. Steps within a phase are sequential. Phases overlap: Phase 1 can run in parallel with Phase 0 once the decision is made, and Phase 3 (hardening) can start in parallel with Phase 2.

If you only read one thing: do every step in Phase 1 regardless of which long-term path you pick.

Phase 0 · Strategic decision

Week 0 · 1 week · ~1 engineer

Before any code lands, decide the customer profile, the compliance ceiling, and the hosting path. The rest of this guide branches based on these answers.

Step 1

Profile your target customer

Who is the user — voice agents, meeting transcription, contact center, healthcare scribe, broadcast captioning? Latency budgets and diarization requirements differ by 5–10× across these.

Write down: peak concurrent streams, P50 latency target, P95 latency target, max session duration, and average audio minutes per tenant per month.

Effort: 0.5 day

Step 2

Set your compliance ceiling

Pick the highest bar you will need within 18 months: SOC 2 Type II only, or HIPAA BAA, or ISO 27001, or FedRAMP Moderate.

This decides whether enterprise-reliability work (Phase 3) is a stretch goal or a launch blocker.

Effort: 0.5 day

Step 3

Choose the hosting path

Cloud-first: proxy to Deepgram + pyannoteAI. Six to ten weeks to 'max each'. Best for small teams. Trades data residency.

Self-hosted: NVIDIA Riva on GPU Kubernetes. Six to nine months. Best for compliance-bound or fine-tuning-heavy customers.

Hybrid: both backends behind one gateway, per-tenant routing. Nine to twelve months. The right end state if you have tenants on both sides of the fence.

Effort: 2–3 days

Step 4

Write the architecture decision record

Single page. Customer profile, latency targets, compliance bar, chosen path, two reasonable alternatives, and the trade-offs you accepted. This becomes the document every engineer refers to when scope creeps.

Effort: 0.5 day

> *Deliverable: a 1-page ADR committed to the repo at* `docs/adr/0001-architecture.md`.

Phase 1 · Quick wins on the current stack

Weeks 1–2 · ~1–2 engineers

Every step below applies regardless of which long-term path you chose in Phase 0. They turn the current single-server FastAPI app into something safer, faster, and easier to operate without changing its architecture.

Step 1

Eliminate the dev API-key default

In `stt_server/config.py`, change `stt_api_key: str = 'dev-secret-key'` to `stt_api_key: str = ''`.

In `main.py`'s lifespan, `raise RuntimeError(...)` if `settings.stt_api_key` is empty and `ENV` is anything other than 'dev'.

Update `.env.example` and the README deployment section to require an explicit value.

Effort: 2 hours

Step 2

Drop the temp-WAV round trip

In `stt_server/model.py`, accept a NumPy float32 array directly: `WhisperModel.transcribe(np_array, ...)`.

In `streaming.py`, convert the PCM-16LE `bytes` to `np.frombuffer(...).astype(np.float32) / 32768.0` instead of writing a WAV.

Measure before/after: time-to-partial should drop 30–50 ms.

Effort: 0.5 day

Step 3

Expose decoder knobs end-to-end

Add optional query params to `WS /stt/stream`: `hotwords` (comma-separated), `initial_prompt`, `beam_size`, `word_timestamps`, `temperature`.

Same parameters on `POST /v1/audio/transcriptions` as form fields.

Pass them through to `WhisperModel.transcribe(...)`. Whisper's `hotwords` is available in faster-whisper ≥ 1.0.

Effort: 1 day

Step 4

Rate limit the REST endpoints

Add `slowapi` or `fastapi-limiter`. Key the limit on the API key (not on IP).

Conservative defaults: 30 transcriptions/minute and 5 admin calls/minute per key. Expose the limit in admin config so tenant tiers can override.

Effort: 0.5 day

Step 5

Structured logging with trace IDs

Replace the JSON line writer in `logging_utils.py` with `structlog` or stdlib JSON.

Generate a trace ID per WebSocket session and per HTTP request. Include it in every log line and surface it in error responses via an `X-Trace-Id` header.

Wire up OpenTelemetry: traces export to OTLP, customer controls the collector endpoint.

Effort: 1 day

Step 6

Cache the API-key map

`auth.get_api_key_map()` currently reparses settings on every request. Wrap it in `functools.lru_cache(maxsize=1)` with a TTL invalidation when keys change.

If you wire in a hashed-key store later, this is the seam.

Effort: 1 hour

Step 7

Ship Phase 1 and measure

Cut a release tag (`v0.2.0`).

Capture a baseline: P50/P95 time-to-first-partial, P50/P95 time-between-partials, WER on a fixed test set with and without hotwords.

These numbers become the bar Phase 2 has to beat.

Effort: 0.5 day

Phase 2 · Pick a path

Two backends, same gateway protocol on the outside. Do 2A if Phase 0 said cloud-first or hybrid. Do 2B if Phase 0 said self-hosted or hybrid. For hybrid, do them in parallel with separate teams.

	Phase 2A — Cloud-first	Phase 2B — Self-hosted
Timeline	3–6 weeks	8–12 weeks
Team	1–2 engineers	3–5 engineers + ML platform
P50 latency	Sub-300 ms (Deepgram Flux)	300–500 ms (Parakeet on Riva)
Diarization	pyannoteAI Precision-2	NVIDIA Sortformer
Custom vocab	Deepgram Keyterm Prompting	NeMo fine-tuning + word boost
Per-minute cost	$0.005–0.02 (Deepgram + pyannoteAI)	Fixed GPU rental
Data residency	US/EU regions only	Full
Vendor lock-in	High	Low

Phase 2A · Cloud-first path

Weeks 3–8 · ~1–2 engineers

Replace the model layer with a proxy to Deepgram (STT) and pyannoteAI (diarization). Keep the existing WebSocket protocol on the client side; transform on the way in and out.

Step 1

Provision provider accounts

Deepgram: production account, Nova-3 or Flux access, region matching your customer base (US-East and EU-West are the usual starting pair).

pyannoteAI: production account, Precision-2 access. Store both API keys in your secrets manager (not in env files).

Effort: 1 day

Step 2

Build the proxy gateway

New module: `stt_server/backends/deepgram.py`. Opens a Deepgram WebSocket, forwards customer audio, translates Deepgram events back into your event schema.

Keep your existing `WS /stt/stream` handler; swap the inner `StreamingTranscriptionSession` for a `DeepgramSession` behind a feature flag (`STT_BACKEND=deepgram|whisper`).

Effort: 1 week

Step 3

Translate the event schema

Deepgram emits `Results` messages with `is_final` and `speech_final`. Map `is_final=false` → `transcript.partial`, `speech_final=true` → `transcript.final`.

Add word-level timestamps and confidences to your event schema now — you'll need them in step 4.

Effort: 2 days

Step 4

Wire up pyannoteAI orchestration

pyannoteAI offers an STT orchestration endpoint that pairs diarization with the transcription backend of your choice. Point it at Deepgram and forward customer audio through it.

Add the `diarization.revise` event type (suggested in the strategy doc) so the client can update earlier speaker labels.

Effort: 1 week

Step 5

Per-tenant keyterm pass-through

Store per-tenant keyterms in a small Postgres table or KV store. On WebSocket open, look up the tenant's keyterms and forward them to Deepgram in the connection URL.

Cap at 100 terms per tenant (Deepgram's limit). Expose a `PUT /v1/tenants/me/keyterms` endpoint for self-service.

Effort: 3 days

Step 6

Load test and cut over

Synthetic load: 100 concurrent streams, 8-hour soak. Compare latency distribution against the Phase 1 baseline.

Roll the feature flag forward in 10% increments per customer tier. Keep the Whisper path live as a fallback for two weeks before retiring it.

Effort: 1 week

Phase 2B · Self-hosted path

Weeks 3–14 · ~3–5 engineers

Stand up the production-grade self-hosted stack: NVIDIA Triton serving Parakeet streaming + Sortformer streaming diarization on a GPU Kubernetes cluster, with state externalized to Postgres and Redis.

Step 1

Provision GPU Kubernetes

EKS / GKE / AKS with an NVIDIA-GPU node pool (L4 for cost, A10G for balance, H100 for top latency). Install the NVIDIA device plugin and DCGM exporter for GPU metrics.

Reserve capacity: streaming ASR cannot tolerate eviction. Use `node-pool autoscaler` with a minimum of 2 GPU nodes for HA.

Effort: 1 week

Step 2

Deploy Triton with Parakeet streaming

Pull NVIDIA's reference Triton config for `parakeet-tdt-streaming` from the NIM catalog. Deploy as a StatefulSet pinned to GPU nodes.

Triton exposes gRPC; your gateway becomes a thin gRPC client. Use Triton's dynamic batching with a `max_queue_delay_microseconds` tuned for your latency budget (start at 5 ms).

Effort: 2 weeks

Step 3

Add Sortformer streaming diarization

Deploy `diar_streaming_sortformer_4spk-v2` alongside Parakeet in the same Triton instance group.

Configure the ASR pipeline so each end-of-utterance gets speaker tags attached at the word level. Surface as `spk_0..spk_3` in your transcript events.

Effort: 1 week

Step 4

Externalize state

Postgres: `tenants`, `api_keys` (hashed), `usage_counters`, `audit_log`. Replace `logs/usage-snapshot.json` with a transactional table.

Redis: ephemeral counters (active connection gauge per pod, per-tenant rate limits). `INCR` on connect, `DECR` on disconnect, `EXPIRE` as a safety net.

Effort: 1 week

Step 5

Connection draining + PodDisruptionBudgets

On `SIGTERM`: stop accepting new WebSocket upgrades, flip readiness to false, wait up to `MAX_SESSION_SECONDS` for in-flight sessions to drain, then exit.

PDB: `maxUnavailable: 1` on the gateway, `minAvailable: 2` on Triton. Set `terminationGracePeriodSeconds` to match max session duration.

Effort: 3 days

Step 6

KEDA autoscaling

Install KEDA. Define a `ScaledObject` for the gateway with `active_websocket_connections` (from Redis) as the trigger.

Separate `ScaledObject` for Triton keyed on `nv_inference_queue_duration_us` from DCGM.

Define `maxReplicaCount` headroom = 2× expected peak.

Effort: 1 week

Step 7

Load test and cut over

Soak test: 200 concurrent streams for 12 hours. Validate that rolling pod restarts don't drop active sessions (they will until step 5 is right — that's the test).

Roll out per tenant tier the same way as Phase 2A. Keep the Whisper path as a fallback during ramp.

Effort: 1 week

Phase 3 · Enterprise hardening

Weeks 9–18 · runs in parallel with Phase 2 · ~2 engineers

These steps are what auditors actually check. None of them are exciting; all of them are non-negotiable for enterprise deals.

Step 1

mTLS at the edge

Terminate TLS at Envoy or an equivalent ingress. Optional mTLS for service-to-service calls into your VPC; required if you advertise PrivateLink-style connectivity.

Cert-manager + Let's Encrypt for public certs; private CA (Smallstep, AWS Private CA, Vault) for internal certs.

Effort: 1 week

Step 2

SSO / SAML / SCIM for tenant admins

Don't build this yourself — integrate with WorkOS, Auth0, or Stytch. Three-day integration vs. three-month rebuild.

Enforce SSO at the org level for any tenant on an enterprise plan. SCIM provisions and deprovisions admin users automatically.

Effort: 1 week

Step 3

RBAC scopes on API keys

Scope keys to operations: `stt:stream`, `stt:transcribe`, `usage:read`, `admin:*`.

Enforce in middleware before the route handler. A key with `stt:stream` only must not be able to hit `/v1/usage/reset`.

Effort: 3 days

Step 4

Per-tenant audit log

Tables: `audit_log(tenant_id, actor_id, event_type, resource, created_at, trace_id, payload_jsonb)`.

Events: key issued/revoked, session started/closed, transcript exported, usage reset, admin login, settings changed.

Per-tenant export endpoint: `GET /v1/admin/audit?tenant_id=...`.

Effort: 1 week

Step 5

Multi-AZ deployment

Spread gateway pods across at least 2 availability zones. Pin Triton replicas to separate node groups in different AZs.

Postgres: managed multi-AZ with automated failover. RPO 5 minutes via PITR. Document the RTO and chaos-test it twice a year.

Effort: 1 week

Step 6

Public status page

Statuspage.io, Better Stack, or Atlassian Statuspage. One component per region per service.

Document SEV taxonomy and postmortem template. Wire alerts from your observability stack into an on-call rotation (PagerDuty, Opsgenie).

Effort: 2 days

Step 7

Begin SOC 2 Type II evidence collection

Pick an evidence-collection platform (Vanta, Drata, Secureframe). Connect your cloud accounts, code repo, ticketing, and HRIS.

SOC 2 Type II is typically 9–12 months to first audit if you're starting from scratch. Start now; finish in Phase 4.

Effort: ongoing

Phase 4 · Differentiation

Months 6–12 · ~2–4 engineers, ML focus

Everything above gets you to parity with the field. Phase 4 is where you build the moat — fine-tuned per-tenant models, speaker identification across sessions, low-latency regional deployments.

Step 1

NeMo fine-tuning recipes (self-hosted only)

Package NVIDIA NeMo's `fine_tune_speech_recognition` recipe as a tenant-facing flow: upload labeled audio, train a Parakeet adapter, deploy automatically.

Charge for fine-tuning as a separate SKU. This is the feature that wins regulated-industry deals.

Effort: 3–6 weeks

Step 2

Domain models

Pre-train models on medical, legal, finance, and contact-center corpora. Surface them as named models in `GET /v1/models`.

Per-tenant default model selection. Each domain typically buys 2–6 WER absolute on in-domain audio.

Effort: 4–8 weeks

Step 3

Speaker enrollment (cross-session identity)

pyannoteAI offers identification; Sortformer alone doesn't. Build the enrollment flow on top: tenant uploads a labeled voice sample, you store an embedding, future sessions resolve `spk_*` to real names.

Privacy: voice embeddings are biometric data. Encrypt at rest with CMK; offer a delete-my-voiceprint API.

Effort: 4 weeks

Step 4

Co-located GPU regions

For latency-sensitive customers, deploy Triton pools in the same region as the customer's app servers. Saves 30–100 ms vs a cross-region path.

Use Anycast or geo-aware DNS for ingress so the WebSocket lands in the closest region automatically.

Effort: 4–6 weeks

Appendix A · Build-vs-buy decision matrix

Capability	Cloud-first wins by	Self-hosted wins by
Raw latency	Sub-300 ms today, no GPU ops	On-prem residency, no per-minute fee
Managed scaling	Thousands of streams out of the box	Predictable cost at high utilization
Diarization	pyannoteAI Precision-2 quality	On-prem, fine-tunable, no third-party data
Reliability	SOC 2 / HIPAA inherited	Full control of incident response
Accuracy tuning	Keyterm Prompting (no retrain)	Full NeMo fine-tuning
Time to ship	6–10 weeks	6–9 months
Engineering team	1–2 engineers	3–5 engineers + ML platform

If forced to pick one row above: most teams should start with cloud-first and add a self-hosted lane only when a specific deal demands it. That is the shortest path to 'max each category' with the lowest engineering bet.

Appendix B · Open questions to answer before shipping

Who is the customer?

Voice agents, meeting transcription, contact center, healthcare scribe, broadcast captioning. Latency budget and diarization quality differ dramatically.

What is the compliance ceiling within 18 months?

SOC 2 Type II is the minimum bar. HIPAA BAA, ISO 27001, FedRAMP Moderate change the Phase 3 scope materially.

Which languages?

English-only favors NVIDIA Parakeet for speed and Canary-Qwen for accuracy. Multilingual favors Whisper-large-v3 or AssemblyAI Universal-3.

What is the cloud egress budget?

Deepgram Nova-3 is roughly $0.0043–0.0058 per audio minute. At 1,000 concurrent streams 8 hours/day that is $80–100k/month. Self-hosted GPU is a fixed cost regardless of utilization.

Can you reserve GPU capacity?

Self-hosted requires sustained access to H100/H200/A100. If you cannot reserve capacity ahead of time, cloud is the lower-risk bet until your spend justifies a commitment.

Appendix C · Sources (May 2026)

Latency benchmarks, leaderboards, and provider feature pages used in this guide. Validate before final architecture commits — the STT field moves quickly.

- Best Speech-to-Text APIs in 2026 — Deepgram — deepgram.com/learn/best-speech-to-text-apis-2026
- Deepgram vs Speechmatics vs AssemblyAI — 2026 Guide — deepgram.com/learn/deepgram-vs-speechmatics-vs-assemblyai
- Best Speech-to-Text Providers in 2026 — Coval benchmarks — coval.ai/blog/best-speech-to-text-providers-in-2026
- Open ASR Leaderboard — Hugging Face — huggingface.co/spaces/hf-audio/open_asr_leaderboard
- Open ASR Leaderboard — Trends and Insights — huggingface.co/blog/open-asr-leaderboard
- NVIDIA Riva ASR Overview — docs.nvidia.com/deeplearning/riva/user-guide/docs/asr
- NVIDIA Streaming Sortformer launch (blog) — developer.nvidia.com/blog/identify-speakers-in-real-time-sortformer
- nvidia/diar_streaming_sortformer_4spk-v2 — Hugging Face — huggingface.co/nvidia/diar_streaming_sortformer_4spk-v2
- pyannoteAI — Real-time diarization platform — pyannote.ai
- pyannoteAI Precision-2 release — pyannote.ai/blog/precision-2
- Deepgram Nova-3 launch — deepgram.com/learn/introducing-nova-3-speech-to-text-api
- Deepgram Keyterm Prompting docs — developers.deepgram.com/docs/keyterm
- Best open-source STT in 2026 — Northflank — northflank.com/blog/best-open-source-speech-to-text-stt-model-in-2026-benchmarks